

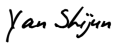
Internship Report Clearance Form

1. Ensure that the form is endorsed by your employer / work supervisor.
2. Attach endorsed form to the front of your report before submission to the Faculty.

Internship Course

- ☐ EG3611A Industrial Attachment (IA) ☐ EG1611 Co-Op Immersion I
- ☒ EG3612 Vacation Industrial Attachment (VIA) ☐ EG2620 Co-Op Immersion II

Student Particulars	
Name	Muthukumaran Yogeeswaran
Matriculation No	A0253028L
Discipline	Engineering Science Programme
Internship Position	Control policy training for multi-robot collaborative manipulation
Internship Report	☐ Interim ☒ Final

Work Supervisor Particulars		
Name	Yan Shijun	
Company	Agency for Science Technology and Research (A*STAR)	
Department	Autonomous Systems & Robotics (ASR)	
Designation	Senior Scientist	
Email / Contact No	yan_shijun@artc.a-star.edu.sg	
<i>The contents of this report have been reviewed and cleared for submission to the NUS College of Design & Engineering for grading.</i>		
Date	Signature	Company Stamp
02 August 2024	Name: Yan Shijun Date: 2024-08-08 14:02:14 +08:00 	



NUS

National University
of Singapore

College of Design and Engineering

Course

☐ EG3611A (IA)

☐ EG1611 (Co-Op)

☒ EG3612 (VIA)

☐ EG2620 (Co-Op)

Report

☐ Interim

☒ Final

Reporting Period

13 May 2024

to

02 Aug 2024

Name

Muthukumaran Yogeeswaran

Matriculation No.

A0253028L

Engineering Discipline

Engineering Science Programme

Internship Period

13 May 2024 - 02 Aug 2024

Mentor

Bettiol, Andrew Anthony

Company

Agency for Science Technology and Research (A*STAR)

Table of Contents

Acknowledgements.....	4
Introduction.....	5
Challenges Addressed.....	6
Adapting URDF model file for MuJoCo software	6
Data fitting for demonstration data	7
Results.....	9
Reflections	11
Conclusion	11
References.....	12
Appendix.....	13
Parameters Configured.....	13
Modified Reward Structure.....	13
Weekly Journal	14

Acknowledgements

I would like to express my deepest gratitude to Yan Shijun, my work supervisor, for giving me the opportunity to work on this project. His guidance and support throughout the duration of this internship have been invaluable.

I am also immensely grateful to my professor, Andrew Anthony Bettiol, for his insightful feedback on my work plan and presentation, and for handling all administrative matters from NUS.

A special thanks to the contributors of the DexMV-Sim GitHub repository (<https://github.com/yzqin/dexmv-sim>). This repository, developed by Yuzhe Qin, Yueh-Hua Wu, Shaowei Liu, Hanwen Jiang, Ruihan Yang, Yang Fu, and Xiaolong Wang, provided the foundation for my project. Using the DexMV system and pipeline, I was able to build upon their work to train a dexterous hand using reinforcement learning. Additionally, I acknowledge the use of the Mujoco simulation software, which was instrumental in the development and testing of my project.

Finally, I would like to extend my thanks to ASTAR for providing me with the opportunity to undertake this internship through the ARIA programme. Their support has been crucial in facilitating my research and professional development.

Introduction

During this internship, I was assigned to work on a project focused on control policy training for multi-robot collaborative manipulation, managed by my supervisor Yan Shijun. My specific role involved developing the control policy for the end effector of the robots, which in this case was a humanoid hand. The objective was to train a dexterous hand to manipulate objects with ease, using human videos as demonstration for the training process.

To kickstart my work, I was provided with a GitHub repository, DexMV-Sim (Qin, 2022) which contained a simulated environment and retargeting code for DexMV, a novel system and pipeline for learning dexterous manipulation from human demonstration videos. DexMV used reinforcement learning to train a model hand called the Adroit Hand as in Figure 1.

My role was to modify the DexMV code so it would work with the Inspire Hand model (INSPIRE-ROBOTS, 2024) as seen in Figure 2. I needed to train a model specifically for the Inspire Hand, ensuring it could effectively manipulate objects using the given simulation and learning framework. I focused on the reallocation task, where the objective was to reallocate an object to a target location that was randomized each time as shown in Figure 3.

This report details the challenges I encountered, and the results of my project in training a dexterous humanoid hand for effective object manipulation.



Figure 1: Adroit hand model

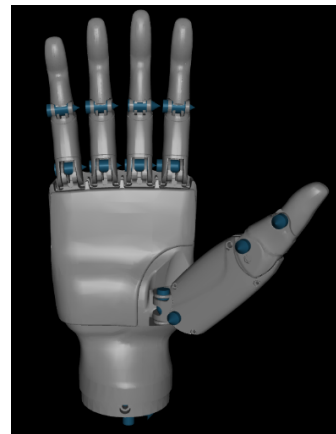


Figure 2: Inspire hand model

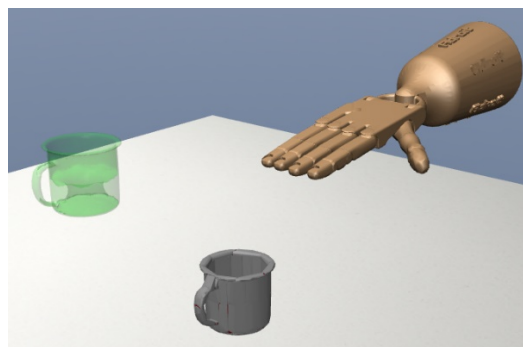


Figure 3: Reallocation task

Challenges Addressed

Adapting URDF model file for MuJoCo software

The DexMV-Sim repository was structured around MuJoCo, a powerful physics engine used for simulations. To utilize the existing code to train my Inspire Hand, I needed to convert the URDF model file into an XML format compatible with MuJoCo's features. MuJoCo provides a tool to assist in converting the URDF file (Figure 4) to an XML file (Figure 5). However, this tool does not produce a complete XML file; it requires the user to manually add missing components such as collision detection, actuator constraints, and joint damping, which are absent in the URDF model file.

```
<robot
  name="inspire_hand_r">

  <link name="world"/>
  <joint name="fixed" type="fixed">
    <origin
      xyz="0.5 0.5 0.5"
      rpy="-1.5707963 0 0" />
    <parent link="world"/>
    <child link="R_hand_base_link"/>
  </joint>

  <link
    name="R_hand_base_link">
    <inertial>
      <origin
        xyz="-0.076223 -0.01701 0.24616"
        rpy="0 0 0" />
      <mass
        value="0.14143" />
      <inertia
        ixx="7.6663E-05"
        ixy="1.6551E-06"
        ixz="1.7709E-06"
        iyy="8.3832E-05"
```

Figure 4: URDF model file of Inspire hand

```
<mujoco model="inspire_hand_r">
  <compiler angle="radian" />
  <option integrator="RK4" timestep="0.001" />
  <size njmax="2000" nconmax="500" nstack="6000" />

  <asset>
    <mesh name="R_hand_base_link" file="..." />
    <mesh name="R_thumb_proximal_base" file="..." />
    <mesh name="R_thumb_proximal" file="..." />
    <mesh name="R_thumb_intermediate" file="..." />
    <mesh name="R_thumb_distal" file="..." />
    <mesh name="R_index_proximal" file="..." />
    <mesh name="R_index_distal" file="..." />
    <mesh name="R_middle_proximal" file="..." />
    <mesh name="R_middle_distal" file="..." />
    <mesh name="R_ring_proximal" file="..." />
    <mesh name="R_ring_distal" file="..." />
    <mesh name="R_pinky_proximal" file="..." />
    <mesh name="R_pinky_distal" file="..." />
  </asset>

  <worldbody>
    <body name="hand" pos="-0.42 -0.75 -0.42">
      <inertial pos="0.42 0.5 0.75" />
      <joint name="ARTx" pos="0.42
```

Figure 5: XML file of Inspire hand

Additionally, I had to group joints together since the Inspire Hand only has six actuators—one for each of the five fingers and one for thumb rotation. Figures 6 and 7 illustrate this, with the actuators in the actual hand circled red in Figure 7.

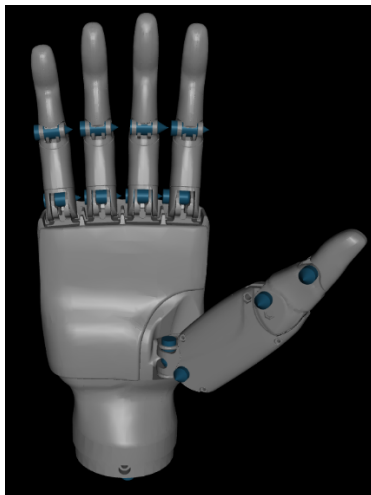


Figure 6: Inspire hand with 12 actuators

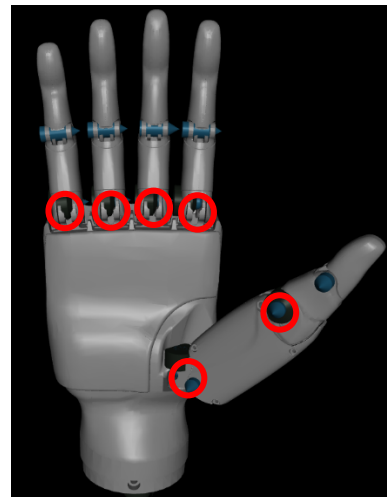


Figure 7: Inspire hand with 6 actuators (circled in red)

Data fitting for demonstration data

The model hand itself trains using two different types of data. Firstly, it learns from human videos, which have been converted into a data format the model can understand, stored in a pickle file as shown in Figure 8.

```
Overview of data loaded from the last batch in the pickle file:  
Variable: model_data, Type: <class 'list'>, List of length 1  
Variable: observations, Type: <class 'numpy.ndarray'>, Numpy array of shape (521, 39), size: 20319  
Variable: rewards, Type: <class 'numpy.ndarray'>, Numpy array of shape (521,), size: 521  
Variable: actions, Type: <class 'numpy.ndarray'>, Numpy array of shape (521, 30), size: 15630  
Variable: sim_data, Type: <class 'list'>, List of length 521
```

87

Figure 8: Pickle file containing demonstration data from Human Videos

Secondly, the model gathers and learns from data collected as it interacts with the environment. The observations from both data sources are fed into the model to train it, and the model outputs actions, which are instructions on how the joints should move to complete a task, as seen in Figure 9. The model outputs 12 actions: the first 6 actions correspond to the hand's x, y, and z movements in the world, along with the pitch, roll, and yaw of the hand. The remaining 6 actions correspond to the 6 actuators of the Inspire Hand.

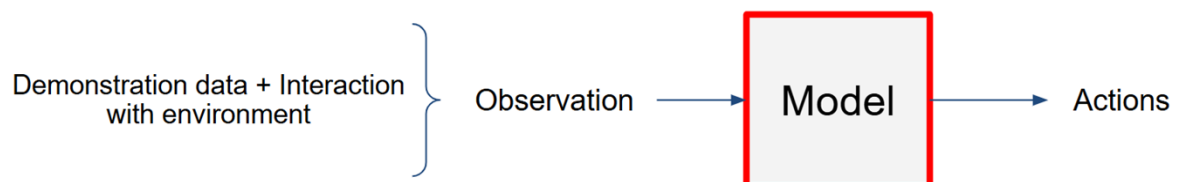


Figure 9: Overview of how the model functions

However, a challenge emerged because the pickle file contained data pre-processed specifically for the Adroit Hand. To adapt this data for training the Inspire Hand, which has a 21-dimensional observation space (12 dimensions for the joints and 9 additional positions for the Hand, Object, and Target), I needed to find a method to utilize the existing data effectively. The 9 additional positions include the Hand's position and orientation, the Object's position, and the Target's position, which help the model assess its performance and adjust its actions accordingly. While these 9 additional positions can be used from the Adroit Hand data, I had to develop a strategy to map the 30 joints of the Adroit Hand to the 12 joints of the Inspire Hand, as shown in Figure 10.

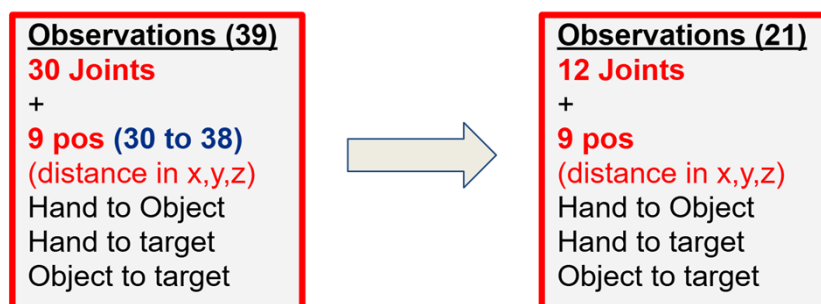


Figure 10: Observation data mismatch

To address the mismatch in joint dimensions, I decided to index the relevant joints as shown in Figures 11 and 12. This approach allowed me to align the demonstration data with the observation space of the Inspire Hand, enabling effective use of the pre-processed data.

```
# Finger with 12DOF
obs_indexes = [0, 1, 2, 3, 4, 5, 9, 13, 17, 22, 26, 28, 30, 31, 32, 33, 34, 35, 36, 37, 38]
act_indexes = [0, 1, 2, 3, 4, 5, 9, 13, 17, 22, 26, 28]
```

Figure 11: Indexes to map adroit hand joints

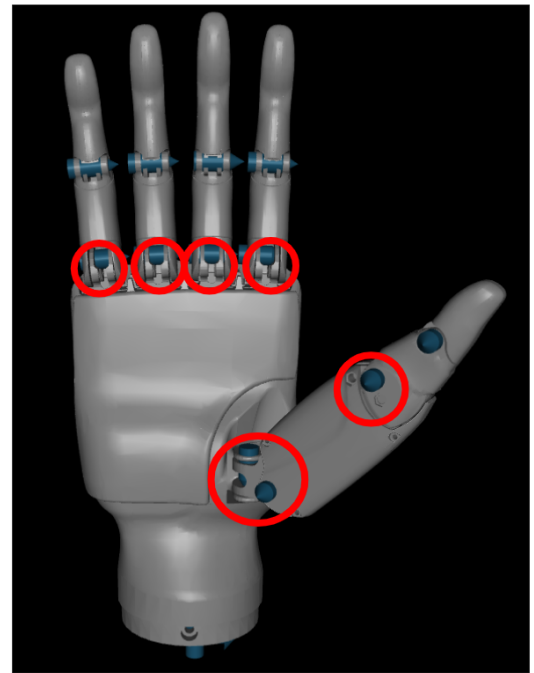
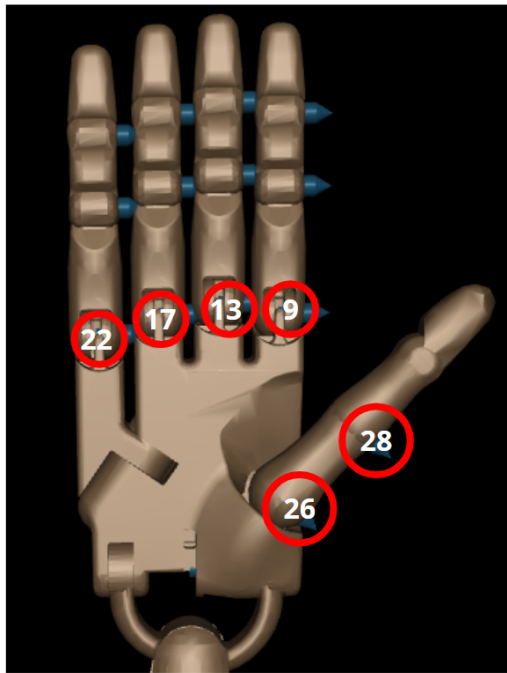


Figure 12: Visual representation of joints mapped

Results

After successfully training the model, new issues emerged. Specifically, the model often tended to reach a local maximum in its reward function, meaning it found an unintended solution to the task. In reinforcement learning, the model earns rewards for desirable actions and penalties for undesirable ones.

The graph in Figure 13 illustrates the reward over time, with the x-axis representing iterations and the y-axis showing the total mean reward. The graph converges at a reward level of 150, indicating that the model reached a point where it could no longer improve its performance.

Further analysis revealed that the model exhibited unexpected behaviour. As shown in Figure 14, instead of directly picking up the mustard bottle, the model first pushes it to the ground before retrieving it and moving it to the target. The intended outcome was for the model to pick up the bottle without it falling, but the model's approach deviated from this expectation.

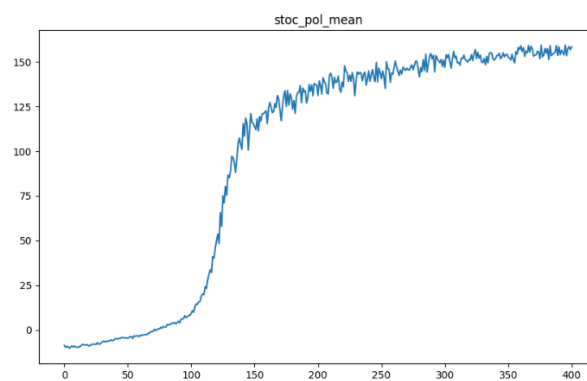


Figure 13: Reward graph of trained model



Figure 14: Trained model on mustard bottle

To improve performance, I adjusted the training parameters to increase the model's reliance on demonstration data, aiming to replicate more human-like behaviour and avoid knocking the bottle down. Additionally, I revised the reward structure to consider the bottle's orientation, which was not accounted for in the initial setup. However, despite these changes, the reward function continued to converge to a negative value, as illustrated in Figure 15. This negative reward was worse than the previous performance, indicating that the model's performance declined rather than improved.

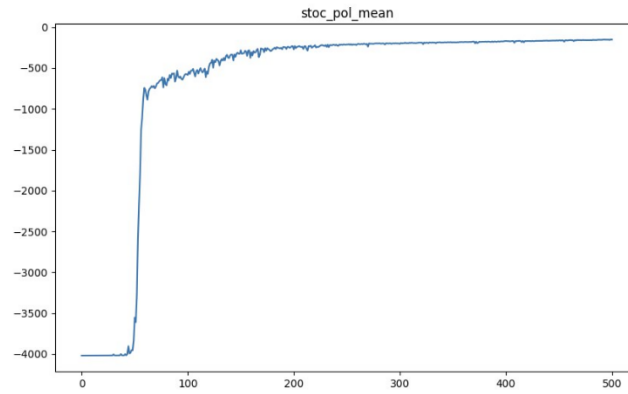


Figure 15: Reward graph of training for modified parameters

From this experience, I learned that simpler reward structures often lead to better outcomes. Consequently, I decided to simplify the task by training the model on a less complex object, such as a mug, rather than the mustard bottle. This adjustment led to significantly improved performance, as shown in Figure 16 and demonstrated in Figure 17.

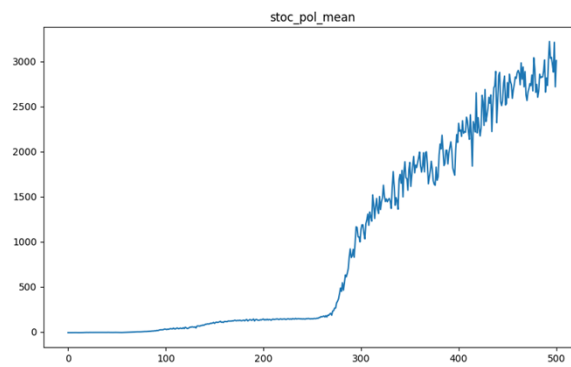


Figure 16: Reward graph of model on mug during training

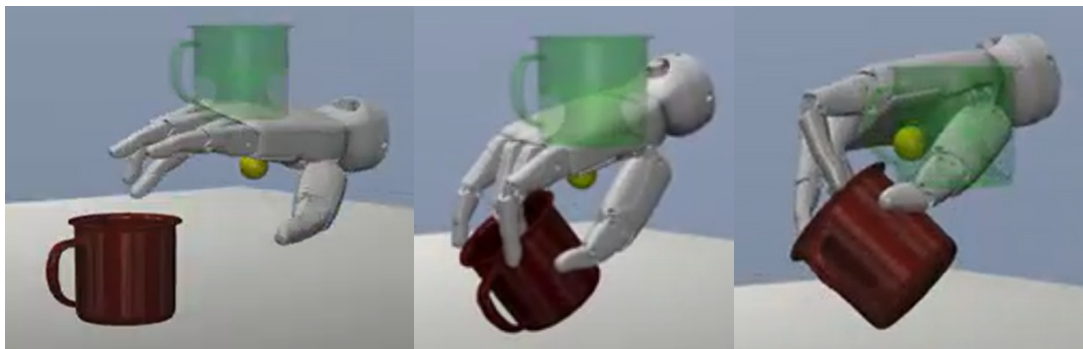


Figure 17: Trained model on mug

Reflections

Through this internship, I acquired many valuable skills, from being exposed to new machine learning techniques like reinforcement learning to developing practical skills like debugging with VSCode and version management in Git. Attending meetings with staff members and higher-ups was an eye-opening experience, where I learned about aspects beyond research, such as project management, budget costs, and even how to file intellectual property rights and patents.

Participating in these meetings with staff and colleagues challenged me to step out of my comfort zone. Introducing myself and engaging with others in these settings was initially nerve-wracking but ultimately valuable for my professional growth. These interactions helped me build confidence, improve my communication skills, and expand my network.

I am especially thankful for choosing a research internship at ARTC in ASTAR, as it allowed me to see the industry side of things as well. I learned that researchers must consider clients and balance many additional factors in their work beyond just scientific research.

During my internship, I encountered several other challenges. One significant challenge was attending weekly progress meetings, where I had to consolidate and present my weekly progress in a concise and structured manner to my supervisor and fellow interns. Even when I hadn't made substantial progress, I learned the value of sharing my struggles. These meetings often provided opportunities to receive feedback and advice from others, which was instrumental in addressing the problems I faced.

Despite the progress made, I recognize areas for improvement. I need to develop better time management skills to ensure more consistent progress. Additionally, enhancing my technical expertise in certain areas, such as advanced reinforcement learning techniques, would help me tackle complex problems more effectively in future projects.

Conclusion

Overall, I have come to realize what a research job entails in Singapore, particularly at A*STAR. I have learned that it is not a one-man job but one that requires collaboration with like-minded individuals and the versatility of a jack-of-all-trades. This is something I didn't expect before experiencing this internship that it would require strong people-handling skills.

Through this experience, my previous belief that a robotics career is not possible in Singapore has been proven false. Observing the innovative projects ASTAR is currently working on, including the exploration of humanoid robots, has broadened my perspective. The organization's forward-thinking approach and commitment to pushing technological boundaries have shown me the exciting potential for a robotics career in Singapore. With these insights, I would consider this as a potential career path in the future.

References

INSPIRE-ROBOTS. (2024). *INSPIRE-ROBOTS*. Retrieved from The Dexterous Hands:
<https://inspire-robots.store/collections/the-dexterous-hands>

Qin, Y. (2022). *DexMV: Imitation Learning for Dexterous Manipulation from Human Videos*. UC San Diego: GitHub.

Appendix

Parameters Configured

```
DEMO_FILE: ../demonstrations/relocate-mug.pkl
ENV_NAME: relocate-mug-0.8
JOB_DIR: ./training_log
JOB_NAME: trpo
POLICY_INIT_LOG_STD: 0.0
POLICY_WS:
  - 32
  - 32
RNG_SEED: 200
NUM_CPU: 10

#Parameters configured
#####
NUM_ITER: 2000
# BC_INIT: true
USE_DAPG: true

DAPG_LAM0: 0.8
DAPG_LAM1: 0.99

SAVE_FREQ: 50
# CHECKPOINT_POLICY: "/home/xiaoyu/Desktop/yogee/dexmv-sim/trained_models_mug/policy_1000.pickle"
# CHECKPOINT_BASELINE: "/home/xiaoyu/Desktop/yogee/dexmv-sim/trained_models_mug/baseline_1000.pickle"
#####
```



Figure 18: Training parameters configured

Modified Reward Structure

```
#####
#-----REWARD STRUCTURE-----#
#####
def reward(self, action):
    obj_pos = self.data.body_xpos[self.obj_bid].ravel()
    palm_pos = self.data.site_xpos[self.S_grasp_sid].ravel()
    target_pos = self.data.body_xpos[self.target_object_bid].ravel()
    is_contact = self.check_contact(self.body_geom_names, self.robot_geom_names)

    reward = -0.1 * np.linalg.norm(palm_pos - obj_pos) # take hand to object

    if is_contact:
        reward += 0.1
        lift = max(min(obj_pos[2], target_pos[2]) - YCB_SIZE[self.object_name][2] / 2.0, 0)
        reward += 50 * lift
        if lift > 0.015: # if object off the table
            obj_target_distance = np.linalg.norm(obj_pos - target_pos)
            reward += 2.0 # bonus for lifting the object
            reward += -0.5 * np.linalg.norm(palm_pos - target_pos) # make hand go to target
            reward += -1.5 * obj_target_distance # make object go to target

            if obj_target_distance < 0.1:
                reward += 1 / (obj_target_distance + 0.01)

    return reward
```

Figure 19: Original reward structure

```
# Penalty for not being upright
upright_penalty = 1 - np.exp(-vertical_angle**2 * 5)
reward -= upright_penalty

# Calculate the vertical angle of the object
obj_quat = self.data.body_xquat[self.obj_bid].ravel()
obj_rot = transforms3d.quaternions.quat2mat(obj_quat)
vertical_angle = np.arccos(obj_rot[2, 1])
```

```
if is_contact:
    # Calculate lift
    lift = max(min(obj_pos[2] - YCB_SIZE[self.object_name][2] / 2.0, 0)
    reward += 50 * lift # Reward for lifting
    # If the object is lifted, reward for proximity to target
    if lift > 0.015:
        obj_target_distance = np.linalg.norm(obj_pos - target_pos)
        reward += 2.0 # Bonus for lifting
        reward += -0.5 * np.linalg.norm(palm_pos - target_pos) # Pen
        reward += -1.5 * obj_target_distance # Penalty for moving o

        if obj_target_distance < 0.1:
            reward += 1 / (obj_target_distance + 0.01) # Higher rew

    # Penalty for not being upright
    upright_penalty = 1 - np.exp(-vertical_angle**2 * 5) # Stronger
    reward -= upright_penalty

# Check if the object is toppled over
toppled_threshold = np.deg2rad(80) # Threshold for considering the
if vertical_angle > toppled_threshold:
    reward -= 10 # Large penalty if the object is toppled over

return reward
```

Figure 20: Modified reward structure

Weekly Journal

The weekly journal should serve as your work log diary. Each entry should highlight one key learning point each week and include a short description. Bullet points are accepted.

You may record your thoughts on how your work tasks contribute to the performance or targets of your business unit. You may also identify perspectives that have changed as a result of your work experiences, including interactions with your managers and colleagues.

Attach this journal to your internship report submission. The journal does not need to be reviewed and endorsed by your workplace supervisor.

Note:

- Remove the rows that are not required.
- If you are absent due to In-Camp Training or other approved reasons, please indicate accordingly with the dates of your absence.

Week No.	Key Learning Point
Week 1	Operated the Inspire-robots dexterous hand using ROS drivers. - Successfully built and ran ROS scripts in ROS noetic to run simulation in Gazebo and Rviz. - Wrote ros2 launch files in ROS2 foxy to move multiple fingers at the same time.
Week 2	Went through GitHub pages on dex-hand-teleop and dexmv-sim . - Installed a software called MuJoCo which is a physics engine used for simulation. - Encountered a lot of issues while running the code and learned how useful Anaconda is for environment management. - Successfully ran the examples in dexmv-sim on my laptop.
Week 3	Understanding the code in dexmv-sim. - Read the project paper on Reinforcement learning and the different algorithms used. - Understood the code structure and how everything came together. Joined my supervisor for an AWS meeting regarding potential collaboration with ARTC. - Learned about the different services offered by them and how it can be applied to our work to significantly speed up the simulation.

Week 4	Modified inspire hand's model to be compatible with MuJoCo. - Read up on MuJoCo documentation to learn how to modify the model's config files. - Converted the original URDF model file into XML file to be compatible with MuJoCo.
Week 5	Worked on Integrating the Inspire hand into the dexmv-sim code. - Leaned to utilize VScode's debug mode to find out why the code couldn't compile.
Week 6	Attempted to train the Inspire hand using the policies from dexmv-sim to complete a reallocation task using different objects. - Using the existing dexmv-sim code, I modified it to be compatible with the Inspire hand model. - Let the model train over a period and assessed its performance using a visualize policy code.
Week 7	Improved the performance of the model. - Continued to modify the Inspire model's file to add properties such as collision constraints to simulate real world physics in MuJoCo. - Started modifying training parameters to improve training effectiveness.
Week 8	Attended ASR (Autonomous Systems & Robots) group meeting. - Got exposure on what scientists in ASTAR do other than research and what other responsibilities they hold such as project management. Started utilizing Git for version control. - Got comfortable in using techniques like branching to improve my coding skills.
Week 9	Attended mindfulness mastery workshop. - Gained some useful skills to help manage work life balance. Modified reward structure for reinforcement learning. - Gained a deeper understanding of what the dexmv-sim repository was doing and got comfortable modifying the code to suit my project needs.

Week 10	Attended SRA Division meeting and ASR monthly meeting. - Was interesting to learn about how the different divisions worked in ASTAR and the future plans they had moving forward such as exploring humanoid robots to stay relevant with the rest of the rising tech industries. - Was introduced to how intellectual properties and patents are filed in ASTAR as well.
Week 11	Improved model file. - To improve training performance of model, I simplified the model by grouping actuators together, so it has lesser actions to learn. - Made further improvements to model file to improve stability in simulation as well.
Week 12	Set up UR5 robot with the dexterous hand. - Learned how to control the UR5 robot. - Helped to set up the robot with the hand so that the next intern can work on integrating the trained model on the hand.